

Field Nation Interview Desk

Backend and system design preparation built around Komol Chandra Devnath's CV. 36 questions with answer points, a 21-day plan and six work-order design cases.

STACK

PHP + MySQL → Node.js
microservices

INFRA

Docker, Kubernetes, AWS

PLUS

RabbitMQ / Kafka, APM, SLI/SLO

FRONTEND

React, Redux, React Native

BAND

BDT 80k–120k / month

HOURS

1–10 PM BD, hybrid, Uttara 12

YOUR ONE-LINE PITCH

“At Sokrio I built a multi-tenant field-force and order platform with GPS check-ins, task tracking, payments and client integrations, and I led its move from a Laravel monolith to event-driven services. Field Nation is solving the same kind of problem at a larger scale.”

01	21-day study plan	3 weeks, must-days marked
02	Self-test checklist	tick shaky or solid
A	Defend your CV	13 questions
B	Backend	20 questions
C	System design	3 questions + 6 cases
03	How to answer	method, numbers, questions to ask

01 21-day study plan

About 3–4 hours on weekdays. If the interview is sooner, do only the **MUST** days in order and drill the checklist daily. Deeper topic lists are in `backend-system-design-study-plan.md`.

Week 1 Defend the CV, then the core backend

DAY	TOPIC	WHAT TO DO	PRIORITY
D1	Domain + CV defense	Learn the work-order lifecycle and marketplace roles. Answer every question in Defend your CV out loud, and write one 90-second story per bullet. Output: <code>cv-stories.md</code> with 10 stories	MUST
D2	HTTP, REST and webhooks	Status codes, idempotency keys, cursor pagination, versioning, HMAC-signed webhooks and retries. Output: OpenAPI sketch for 4 work-order endpoints	MUST
D3	Node.js runtime + TypeScript	Event-loop phases, microtasks vs nextTick, blocking the loop, streams, Promise combinators. TS generics, unknown, discriminated unions. Output: 10 answers in your own words	MUST
D4	NestJS by building	Build a small <code>work-order-service</code> : module, controller, service, DTO validation, guard, TypeORM + MySQL, global exception filter. Output: GitHub repo you can show	MUST
D5	MySQL indexing	Composite index order, leftmost prefix, covering indexes, reading EXPLAIN (type, key, rows, Extra), keyset pagination. Output: 5 slow queries fixed with a before/after EXPLAIN	MUST
D6	MySQL transactions + concurrency	Isolation levels, REPEATABLE READ and next-key locks, SELECT ... FOR UPDATE vs optimistic versioning, deadlocks, zero-downtime migrations. Output: Race-safe 'assign technician' query	MUST
D7	Review + mock 1	Mock round on the CV and REST questions. Redo every card marked shaky. Output: List of weak cards	REVIEW

Week 2 Distributed backend

DAY	TOPIC	WHAT TO DO	PRIORITY
D8	Caching with Redis	Cache-aside, TTLs, invalidation, stampede protection, tenant-scoped keys, what never to cache. Output: Cache layer added to the repo	HIGH
D9	Queues + event-driven	RabbitMQ exchanges/queues/acks, DLQ, at-least-once delivery, idempotent consumers, the outbox pattern, RabbitMQ vs Kafka. Output: WorkOrderAssigned event → notification consumer	MUST
D10	Microservices	Service boundaries, sync vs async calls, sagas, the strangler fig pattern, API gateway, data ownership. Output: Diagram of your Sokrio migration, redrawn cleanly	MUST
D11	Docker, Kubernetes, AWS	Multi-stage Dockerfile, Pod/Deployment/Service/Ingress, probes, SIGTERM handling, HPA, ConfigMap/Secret. Map these to ECS/EKS, RDS, SQS. Output: Repo running on minikube or kind	HIGH
D12	Observability + SLOs	SLI vs SLO vs SLA, error budgets, the four golden signals, RED/USE, tracing with correlation IDs, alerting on burn rate. Output: One written SLO for the work-order API	HIGH
D13	Testing + TDD	Red-green-refactor, test pyramid, Jest unit and supertest e2e, mocking repositories, contract tests between services. Output: One endpoint rebuilt test-first	HIGH
D14	Mock 2	Full backend round: 10 random cards with a 3-minute limit each. Output: Scores in this page	REVIEW

Week 3 System design

DAY	TOPIC	WHAT TO DO	PRIORITY
D15	Framework + estimation	Practise the 45-minute framework below. Do back-of-envelope estimates from Field Nation's real scale. Output: One-page cheat sheet	MUST
D16	Case: dispatch + matching	Design routing a work order to technicians. Cover the assignment race and geo search. Output: Diagram + trade-off list	MUST
D17	Case: notifications + webhooks	Multi-channel notifications, and outbound webhooks to buyer systems. Output: Diagram	MUST
D18	Case: check-in and offline mobile	GPS check-in/out, offline sync, photo deliverables, anti-spoofing. Output: Diagram	HIGH
D19	Case: approval + payouts	Ledger, idempotent payouts, reconciliation, the state machine. Output: Diagram	HIGH
D20	Case: monolith → Node service	Strangler migration of one PHP module with zero downtime. Output: Cutover runbook	MUST
D21	Final mock	One design case plus 8 random cards. Rest in the evening. Output: Ready	REVIEW

02 Self-test checklist

Answer each question out loud before you read its answer points. Tick Shaky or Solid, and redo the shaky ones on the review days.

Part A • Defend your CV		
CV-01	You say 'zero cross-tenant data leakage across 100+ tenants.' How did you enforce that, and how do you know it is true?	☐ Shaky ☐ Solid
CV-02	Walk me through the monolith to microservices migration. Why split, what did you split, and how was it zero-downtime?	☐ Shaky ☐ Solid
CV-03	How did RabbitMQ fit in? Explain your exchanges, queues, retries and dead-letter setup.	☐ Shaky ☐ Solid
CV-04	'Report load times cut by up to 85%.' Take one report and show me before and after.	☐ Shaky ☐ Solid
CV-05	You tracked a 99.9% SLO for report generation. Define the SLI, how you measured it, and what the error budget was.	☐ Shaky ☐ Solid
CV-06	How exactly did you cut issue resolution time by 65%? What was your on-call workflow?	☐ Shaky ☐ Solid
CV-07	Your payment integration had 'zero failed transactions.' How do HMAC verification and idempotency keys actually work?	☐ Shaky ☐ Solid
CV-08	How did you detect GPS spoofing?	☐ Shaky ☐ Solid
CV-09	Tell me about the Node.js services you built. What did they do, and how were they structured?	☐ Shaky ☐ Solid
CV-10	Show me a MySQL optimization you did: the query, the index, and the EXPLAIN before and after.	☐ Shaky ☐ Solid
CV-11	How did your CI/CD make deployments 40% faster, and what does the pipeline look like?	☐ Shaky ☐ Solid
CV-12	Your skills say 'Kubernetes (familiar)'. What have you actually done with it?	☐ Shaky ☐ Solid
CV-13	You write tests first. Show me how you would TDD a small feature right now.	☐ Shaky ☐ Solid
Part B • Backend		
BE-01	A client's POST /work-orders timed out, they retried, and now there are two work orders. How do you prevent this?	☐ Shaky ☐ Solid
BE-02	PUT vs PATCH vs POST: which are idempotent? And 401 vs 403, 400 vs 422, 409?	☐ Shaky ☐ Solid
BE-03	Why is OFFSET pagination slow on 10M rows, and how does cursor (keyset) pagination fix it?	☐ Shaky ☐ Solid
BE-04	Design webhooks so a buyer's system can trust them, deduplicate them, and survive our retries.	☐ Shaky ☐ Solid

BE-05	Explain the Node.js event loop. In what order do setTimeout, setImmediate, Promise.then and process.nextTick run?	☐ Shaky ☐ Solid
BE-06	Promise.all vs Promise.allSettled vs Promise.race vs Promise.any. And how do errors escape in async Express handlers?	☐ Shaky ☐ Solid
BE-07	Walk through the NestJS request lifecycle and map it to Laravel.	☐ Shaky ☐ Solid
BE-08	Model the work-order status in TypeScript so invalid transitions don't compile or don't run.	☐ Shaky ☐ Solid
BE-09	Index (buyer_id, status, created_at). Which queries use it well, and which don't?	☐ Shaky ☐ Solid
BE-10	Two dispatchers assign the same work order to different technicians at the same moment. How do you make only one win?	☐ Shaky ☐ Solid
BE-11	Explain MySQL isolation levels and which anomalies each allows.	☐ Shaky ☐ Solid
BE-12	How do you add a column or index to a 50M-row table in production without downtime?	☐ Shaky ☐ Solid
BE-13	Where do you put Redis caching, how do you invalidate, and how do you stop a cache stampede?	☐ Shaky ☐ Solid
BE-14	What is the transactional outbox pattern and why do you need it?	☐ Shaky ☐ Solid
BE-15	RabbitMQ vs Kafka: when would you choose each?	☐ Shaky ☐ Solid
BE-16	Service A calls service B synchronously and B is slow. What protects A?	☐ Shaky ☐ Solid
BE-17	Liveness vs readiness probes, and how does a Node service shut down gracefully in Kubernetes?	☐ Shaky ☐ Solid
BE-18	What are the four golden signals, and how would you trace one request across three services?	☐ Shaky ☐ Solid
BE-19	How do you test a service that writes to MySQL and publishes to RabbitMQ?	☐ Shaky ☐ Solid
BE-20	API authentication for third-party integrations: API keys, OAuth2 client credentials, or JWT?	☐ Shaky ☐ Solid
Part C • System design		
SD-01	Estimate the load: 1M work orders a year, 100k technicians. What dominates the traffic?	☐ Shaky ☐ Solid
SD-02	Monolith or microservices for a new feature? How do you decide service boundaries?	☐ Shaky ☐ Solid
SD-03	How does a saga handle 'approve work → charge buyer → pay technician' across services?	☐ Shaky ☐ Solid

A Defend your CV

Every claim on your CV can become a question. Answer each one out loud in about two minutes with a real example and a number, then compare with the points below. Where your own facts differ from the points, trust your facts.

CV-01 You say 'zero cross-tenant data leakage across 100+ tenants.' How did you enforce that, and how do you know it is true?

Your CV says Built landlord/tenant database isolation, tenant-scoped queries and territory-scoped RBAC ... zero cross-tenant data leakage

They want: mechanism, not a slogan. 'Zero' claims always get a 'how do you know?' follow-up.

ANSWER POINTS

- Isolation model: landlord DB for platform data, one DB per tenant. The tenant is resolved from the request (domain/header), and the connection is switched in middleware before any controller runs.
- Defence in depth: switch back to the landlord connection in a finally block, include the tenant ID in every cache key, and scope queue jobs to carry the tenant ID and reconnect on the worker.
- Territory RBAC sits on top: the permission check says what you may do, and the territory scope limits which rows you may see.
- How you know: feature tests that create two tenants and assert that tenant A's token cannot read B's data; no incident reports; audit of cache keys and jobs. Say 'no leak reported or found in audits', which is more defensible than 'zero'.

LIKELY FOLLOW-UPS

- What happens to a queued job if the tenant context is lost?
- DB-per-tenant vs shared schema with tenant_id: what are the trade-offs at 10,000 tenants?
- How do you run a migration across 100+ tenant DBs?

Tip Bring up the job and cache scoping. Most candidates only mention the DB layer, so it shows you thought about every path data takes.

CV-02 Walk me through the monolith to microservices migration. Why split, what did you split, and how was it zero-downtime?

Your CV says Led the migration of a Laravel 8/Vue.js 2 monolith to an event-driven microservices platform ... zero-downtime cutover, cutting server load by about 20%

This is your headline story. Field Nation is doing the same move (PHP → Node), so expect 15+ minutes on it.

ANSWER POINTS

- Why: reports and notifications were slowing the core API. They had different scaling needs and release cycles.
- What: kept the Laravel core REST API, and extracted Report, Notification and SOP services. They communicate over RabbitMQ, use MongoDB for heavy read models and Redis for cache, with separate React tenant and admin apps.
- How (strangler fig): new services consumed events published by the core. You ran old and new paths in parallel and compared outputs, then moved traffic tenant by tenant or route by route with a rollback switch.
- Result: about 20% lower server load, because heavy reads moved off the core DB.

- What you would do differently: an outbox table from day one, contract tests between services, and tracing IDs across the queue.

LIKELY FOLLOW-UPS

- How did you keep data consistent while both systems ran?
- What if an event was published but the DB transaction rolled back?
- Which service did you decide NOT to extract, and why?

Tip Say 'I would do this differently next time.' Interviewers trust a candidate who names a mistake.

CV-03 How did RabbitMQ fit in? Explain your exchanges, queues, retries and dead-letter setup.

Your CV says Report, Notification and SOP services communicating over RabbitMQ ... Laravel queue jobs (retries, dead-letter queues, priority jobs)

They want: proof you used it, not just read about it. Know the vocabulary exactly.

ANSWER POINTS

- Producer → exchange (direct/topic/fanout) → binding → queue → consumer. Topic exchange with routing keys like `order.created` lets several services subscribe.
- Manual acks: ack only after processing succeeds. nack or requeue on failure, and cap retries with a retry count header or a delayed retry queue (TTL + dead-letter back to the main queue).
- DLQ: set `x-dead-letter-exchange` on the queue. Messages that fail N times land there, you get an alert, and you can replay them after a fix.
- Prefetch (QoS) limits in-flight messages per consumer so one slow worker is not flooded.
- Delivery is at-least-once, so consumers must be idempotent: store processed message IDs with a unique key.

LIKELY FOLLOW-UPS

- How do you guarantee ordering for one order's events?
- When would you pick Kafka instead?
- What happens if a consumer crashes after doing the work but before the ack?

Tip Be clear which parts used Laravel queues (Redis driver) and which used RabbitMQ between services. Mixing them up is the fastest way to lose credibility.

CV-04 'Report load times cut by up to 85%.' Take one report and show me before and after.

Your CV says heavy reports over millions of records using MongoDB aggregation pipelines, Laravel queue jobs ... Redis caching, cutting report load times by up to 85%

They want: a real diagnosis story with numbers. 'Up to' will get probed.

ANSWER POINTS

- Pick one report: name the data size, the old time (for example 40 s), and the new time (6 s).
- Diagnosis: slow query log / Telescope → EXPLAIN → find a full scan or filesort, or N+1 in the loop.
- Fixes in order: composite index matching WHERE + ORDER BY, remove N+1 with eager loading, precompute summaries (delivery_summaries / order_summaries in MongoDB), move generation to a queue job for huge exports, cache results per tenant + filter hash.
- State how you measured: p95 before and after, not a single run.

LIKELY FOLLOW-UPS

- How do you keep precomputed summaries correct when orders change?
- What is your cache invalidation rule for that report?

Tip Have exact numbers for one report ready. A precise small number beats a vague big one.

CV-05 You tracked a 99.9% SLO for report generation. Define the SLI, how you measured it, and what the error budget was.

Your CV says tracked SLIs/SLOs (99.9% report-generation availability, API latency and error rate) ... SLO-based alerts

The job post names SLI/SLO directly. Expect them to check whether you know the maths.

ANSWER POINTS

- SLI = good events / valid events. Example: report jobs finished without error within 60 s ÷ report jobs requested.
- SLO = target for the SLI over a window: 99.9% over 30 days.
- Error budget = 0.1% of events, or in time terms 43.2 minutes a month ($30 \times 24 \times 60 \times 0.001$).
- Alerting on burn rate: page when the budget is burning fast (for example 14x over 1 hour), open a ticket when it burns slowly. This beats alerting on every 5xx.
- SLA is the external contract with penalties and is looser than the SLO.

LIKELY FOLLOW-UPS

- What do you do when the error budget is used up?
- Which SLI would you choose for the work-order API?

Tip Say the 43.2-minute number. It shows you have done the calculation.

CV-06 How exactly did you cut issue resolution time by 65%? What was your on-call workflow?

Your CV says Primary 24/7 on-call engineer for six production apps ... cut issue resolution time by 65%

They want: a process, plus one real incident told end to end.

ANSWER POINTS

- Before: users reported problems, and you searched raw logs by hand.
- After: structured logs with tenant_id and request_id, dashboards per app, alerts before users noticed, and runbooks for recurring issues.
- Metric: mean time to resolve from ticket timestamps, before vs after.
- Tell one incident: symptom → alert → hypothesis → isolate → fix → prevention (test + alert + runbook).

LIKELY FOLLOW-UPS

- What was the worst outage you handled?
- How do you avoid alert fatigue?

Tip Prepare one incident story, timed at 2 minutes.

CV-07 Your payment integration had 'zero failed transactions.' How do HMAC verification and idempotency keys actually work?

Your CV says initiation, webhooks, retries, auto-reconciliation and receipts, secured with HMAC signature verification and idempotency keys

Field Nation pays technicians. Payment correctness questions are very likely.

ANSWER POINTS

- HMAC: provider signs `timestamp + raw body` with a shared secret (HMAC-SHA256). You recompute over the raw bytes, compare with a constant-time function, and reject old timestamps to block replays.
- Idempotency: every payment attempt carries a unique key. A unique constraint on (key) means a retried request returns the stored result instead of charging twice.
- Webhook handling: store the event ID with a unique index, respond 200 fast, process in a job. Duplicate events become no-ops.
- Reconciliation: a scheduled job compares your records with the gateway's settlement report and flags mismatches.
- Rephrase 'zero failed' as 'no double charges or lost payments'. Declines still happen, and an interviewer will point that out.

LIKELY FOLLOW-UPS

- The gateway charged the card but your server crashed before saving. What happens?
- Why must HMAC be computed on the raw body and not on parsed JSON?

Tip Answer the crash scenario with: pending record before the call, idempotency key, then webhook + reconciliation fix the state.

CV-08 How did you detect GPS spoofing?

Your CV says Van Sales, Vehicle Feedback and Van Attendance modules ... plus GPS-spoofing fraud detection

Field Nation checks technicians in by GPS. This is directly relevant, so give a concrete answer.

ANSWER POINTS

- Client signals: Android mock-location flag, developer options enabled, known spoofing apps.
- Server checks: impossible travel speed between consecutive points, location accuracy that is suspiciously perfect, timestamps out of order, check-in outside the outlet's geofence radius.
- Response: flag for review rather than auto-block, to limit false positives, and track the flag rate per user.
- Name which of these you actually built. Don't claim ones you didn't.

LIKELY FOLLOW-UPS

- How would you handle a technician in a basement with poor GPS?
- How do you store and query location history efficiently?

Tip Connect it to Field Nation's check-in / check-out flow. It is the same problem.

CV-09 Tell me about the Node.js services you built. What did they do, and how were they structured?

Your CV says Built Node.js/Express services, background workers and developer tooling alongside the Laravel core API, in JavaScript and TypeScript

Your biggest risk. Node is their target stack and your CV names it only generally. Expect deep follow-ups on the event loop and error handling.

ANSWER POINTS

- Name one concrete service: its purpose, its inputs and outputs, and how it talks to Laravel (REST or queue).
- Structure: routes → controllers → services → data layer, central error middleware, env config validation, structured logging.
- Runtime facts: single-threaded event loop, keep CPU-heavy work off it (worker_threads or a separate job), handle unhandledRejection, graceful shutdown on SIGTERM.
- If NestJS is still 'learning', say so and show your practice repo. Being honest and showing evidence beats bluffing.

LIKELY FOLLOW-UPS

- How do you handle an error thrown inside an async Express handler?
- How did you share types between the Node service and the frontend?

Tip The D4 NestJS repo exists for this question. Have the link ready.

CV-10 Show me a MySQL optimization you did: the query, the index, and the EXPLAIN before and after.

Your CV says Tuned MySQL with composite indexes, EXPLAIN analysis and N+1 elimination

MySQL is their core DB. They will ask you to read or write an index.

ANSWER POINTS

- Query shape: `WHERE tenant_scope AND status = ? AND created_at BETWEEN ? AND ? ORDER BY created_at`.
- Index: equality columns first, then the range/sort column, for example `(status, created_at)`.
- EXPLAIN before: `type=ALL`, rows $\approx$ 2M, `Using filesort`. After: `type=range`, rows $\approx$ 8k, no filesort.
- N+1: 1 query for 500 orders + 500 queries for customers becomes 2 queries with eager loading.

LIKELY FOLLOW-UPS

- Why does index (a, b) not help WHERE b = ?
- When does adding an index make things slower?

Tip Draw the index on the whiteboard as a sorted list, then show why the column order matters.

CV-11 How did your CI/CD make deployments 40% faster, and what does the pipeline look like?

Your CV says Containerized services with Docker and built CI/CD pipelines ... deployments about 40% faster

Docker + K8s are in the post. Deployment detail links to that.

ANSWER POINTS

- Stages: lint → unit tests → build image (multi-stage, cached layers) → push with commit SHA tag → deploy → migrate → health check → roll back on failure.

- Faster because of: layer caching, parallel test jobs, building once and promoting the same image across environments.
- Zero-downtime: new containers must pass health checks before old ones are removed, and migrations are backward-compatible (expand, then contract).

LIKELY FOLLOW-UPS

- How do you roll back a deploy that included a DB migration?
- Where do secrets live?

Tip Mention the expand/contract migration pattern. It shows production maturity.

CV-12 Your skills say 'Kubernetes (familiar)'. What have you actually done with it?

Your CV says Docker, Docker Compose, CI/CD, AWS ... Kubernetes (familiar)

An honesty check. Field Nation runs services on Kubernetes.

ANSWER POINTS

- Be exact: 'I ran my practice service on minikube/kind with a Deployment, Service, probes and ConfigMap. Production at Sokrio was Docker Compose on servers.'
- Then show you know the concepts: Pod, Deployment (rolling update), Service, Ingress, liveness vs readiness probe, HPA, requests/limits.
- Close with how you learn fast: the Vue → React rewrite you led in 2 months.

LIKELY FOLLOW-UPS

- Liveness vs readiness: what happens if you mix them up?
- A pod keeps restarting. How do you debug it?

Tip Clear limits plus a practice demo come across as senior. Don't bluff here.

CV-13 You write tests first. Show me how you would TDD a small feature right now.

Your CV says Wrote tests first for new service and API logic with PHPUnit and pytest

They may ask for live TDD. Practise the rhythm out loud.

ANSWER POINTS

- Red: write the smallest failing test (for example 'cannot assign a cancelled work order').
- Green: write the least code to make it pass.
- Refactor: clean up while tests stay green, then add the next test.
- Explain the pyramid: many unit tests, fewer integration tests, few e2e. Mock at the boundaries (DB repository, HTTP clients), not inside your own logic.

LIKELY FOLLOW-UPS

- When is TDD not worth it?
- How do you test code that calls RabbitMQ?

Tip Use the work-order state machine as your kata. It is small and relevant.

B Backend

The fundamentals Field Nation's stack depends on: REST, Node.js, TypeScript, MySQL, caching, messaging, Kubernetes, observability and testing.

BE-01 A client's POST /work-orders timed out, they retried, and now there are two work orders. How do you prevent this?

Integrations are core to Field Nation. Buyers create work orders from their own ticketing tools.

ANSWER POINTS

- Client sends an `Idempotency-Key` header (a UUID per logical request).
- Server stores key + request hash + response in a table with a unique index on (client_id, key).
- Same key + same hash → return the saved response. Same key + different body → 422. Same key still in progress → 409 or wait.
- Keys expire after, say, 24 hours.
- A natural unique key such as (client_id, external_ticket_id) is a second safety net.

LIKELY FOLLOW-UPS

- Where do you store keys: Redis or MySQL? What if Redis evicts one?

BE-02 PUT vs PATCH vs POST: which are idempotent? And 401 vs 403, 400 vs 422, 409?

A quick fundamentals check. Answer in under 60 seconds.

ANSWER POINTS

- GET, PUT and DELETE are idempotent. POST is not. PATCH is not guaranteed: 'set status=done' is idempotent, 'append a note' is not.
- 401: not authenticated (no token or bad token). 403: authenticated but not allowed.
- 400: malformed request. 422: well-formed but fails validation.
- 409: conflicts with current state (for example assigning an already assigned work order). 429: rate limited, send Retry-After.

LIKELY FOLLOW-UPS

- Which code do you return for an async job you accepted but haven't finished? (202)

BE-03 Why is OFFSET pagination slow on 10M rows, and how does cursor (keyset) pagination fix it?

Work-order lists are large. This is a common MySQL + API question.

ANSWER POINTS

- OFFSET 500000 makes MySQL read and throw away 500k rows. Cost grows with page depth.
- Keyset: `WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 50` with an index on (created_at, id). Each page costs the same.
- Return an opaque cursor (base64 of the last row's values).
- Trade-off: no 'jump to page 57', and it needs a stable, unique sort key.

LIKELY FOLLOW-UPS

- How do you add filters (status, buyer) and keep it fast?

BE-04 Design webhooks so a buyer's system can trust them, deduplicate them, and survive our retries.

Direct Field Nation scenario: work-order status webhooks to buyer systems.

ANSWER POINTS

- Payload has event_id, event type, occurred_at, and the resource ID or a snapshot.
- Signature: HMAC-SHA256 over timestamp + body in a header. The receiver checks it in constant time and rejects old timestamps.
- Delivery: at-least-once. Retry with exponential backoff + jitter for about 24 h, then mark failed and show it in the buyer's dashboard with a replay button.
- Receivers dedupe by event_id. Ordering is not guaranteed, so include a version or updated_at, or tell receivers to fetch the latest state.
- Isolation: a per-endpoint queue, so one slow buyer does not delay the others. Disable endpoints that keep failing.

LIKELY FOLLOW-UPS

- How would you let a buyer rotate the signing secret without downtime?

BE-05 Explain the Node.js event loop. In what order do setTimeout, setImmediate, Promise.then and process.nextTick run?

The most common Node question. They are moving to Node.

ANSWER POINTS

- One JS thread. I/O is offloaded to the OS or the libuv thread pool, and callbacks come back to the loop.
- Phases: timers → pending callbacks → poll (I/O) → check (setImmediate) → close callbacks.
- After each callback: the process.nextTick queue drains first, then the Promise microtask queue.
- So in the main module: sync code → nextTick → promise → then timeout vs immediate (order not guaranteed in the main module; inside an I/O callback, setImmediate comes first).
- Blocking the loop (big JSON.parse, sync crypto, heavy loops) stalls every request. Fix with worker_threads, chunking, or moving the work to a queue.

LIKELY FOLLOW-UPS

- What does the libuv thread pool handle? (fs, dns.lookup, crypto, zlib)

BE-06 Promise.all vs Promise.allSettled vs Promise.race vs Promise.any. And how do errors escape in async Express handlers?

Everyday Node correctness.

ANSWER POINTS

- all: rejects at the first rejection. allSettled: waits for all and returns each result. race: first to settle wins. any: first to fulfil wins, and rejects only if all reject.
- Express 4 does not catch rejected promises from async handlers. Wrap them or call next(err). Express 5 and NestJS handle it for you.
- Always listen for unhandledRejection and log it. Don't keep running in an unknown state.

LIKELY FOLLOW-UPS

- How would you limit concurrency to 5 parallel calls?

BE-07 Walk through the NestJS request lifecycle and map it to Laravel.

Shows you can move from Laravel to Nest quickly.

ANSWER POINTS

- Order: middleware → guards → interceptors (before) → pipes → controller → service → interceptors (after) → exception filters on error.
- Laravel mapping: middleware = middleware, guard ≈ Gate/Policy or auth middleware, pipe + DTO/class-validator ≈ FormRequest, interceptor ≈ response macro or middleware after-hook, exception filter ≈ Handler::render, provider/DI ≈ service container.
- Modules group controllers + providers, like Laravel service providers per domain.

LIKELY FOLLOW-UPS

- Where would you put tenant resolution in Nest?

BE-08 Model the work-order status in TypeScript so invalid transitions don't compile or don't run.

Tests TS depth (discriminated unions) on their core entity.

ANSWER POINTS

- `type Status = 'draft' | 'published' | 'assigned' | 'checked_in' | 'done' | 'approved' | 'paid' | 'cancelled'`
- Transition map: `const next: Record<Status, Status[]> = { draft: ['published', 'cancelled'], ... }` and a function that throws a domain error (→ 409) when the move is not allowed.
- A discriminated union per state (`{status: 'assigned', providerId: string}`) makes the compiler require fields that only exist in that state.
- Enforce it in the DB too, with a conditional UPDATE (see BE-10).

LIKELY FOLLOW-UPS

- unknown vs any? When would you use a generic here?

BE-09 Index (buyer_id, status, created_at). Which queries use it well, and which don't?

Advanced SQL is often tested this way.

ANSWER POINTS

- Works well for: `buyer_id = ?`; `buyer_id = ? AND status = ?`; `buyer_id = ? AND status = ? ORDER BY created_at`; `buyer_id = ? AND status = ? AND created_at > ?`.
- Partial: `buyer_id = ? AND created_at > ?` uses only the `buyer_id` part for seeking.
- Not usable for a seek: `status = ?` alone, or `created_at` alone (leftmost prefix rule).
- A range on a middle column stops the columns after it from being used for seeking.
- Covering index: if every selected column is in the index, EXPLAIN shows `Using index` and the table is never read.

LIKELY FOLLOW-UPS

- Read an EXPLAIN: what do `type=ref`, `type=range`, `type=ALL` and `Using filesort` mean?

BE-10 Two dispatchers assign the same work order to different technicians at the same moment. How do you make only one win?

The classic concurrency question for a marketplace.

ANSWER POINTS

- Conditional update (optimistic): `UPDATE work_orders SET provider_id=?, status='assigned', version=version+1 WHERE id=? AND status='published'`. If 1 row changed you won; if 0, return 409.
- Or pessimistic: `SELECT ... FOR UPDATE` inside a transaction, check the status, update, commit. Keep the transaction short.
- InnoDB default is REPEATABLE READ. A plain SELECT then UPDATE without a lock is a race.
- Publish the `WorkOrderAssigned` event after commit (outbox), never before.

LIKELY FOLLOW-UPS

- What is a deadlock and how do you reduce them? (same lock order, short transactions, retry)

BE-11 Explain MySQL isolation levels and which anomalies each allows.

Senior-leaning, but it appears in mid-level rounds too.

ANSWER POINTS

- READ UNCOMMITTED: dirty reads. READ COMMITTED: no dirty reads, but non-repeatable reads. REPEATABLE READ (InnoDB default): stable snapshot per transaction; locking reads use next-key locks to block phantoms. SERIALIZABLE: plain SELECTs become locking reads.
- Snapshot (MVCC) reads vs locking reads (FOR UPDATE / FOR SHARE) behave differently. That difference is the usual trap.

LIKELY FOLLOW-UPS

- Why might you run reports on READ COMMITTED or on a replica?

BE-12 How do you add a column or index to a 50M-row table in production without downtime?

You ran 100+ tenant DBs. They will expect this.

ANSWER POINTS

- MySQL 8 online DDL (`ALGORITHM=INSTANT` for many column adds, `INPLACE` for indexes), or `gh-ost` / `pt-online-schema-change` for big tables.
- Expand → migrate → contract: add a nullable column, deploy code that writes to both, backfill in batches, switch reads, then remove the old column.

- Every migration must work with both the old and the new app version during a rolling deploy.

LIKELY FOLLOW-UPS

- How do you backfill without locking or replica lag?

BE-13 Where do you put Redis caching, how do you invalidate, and how do you stop a cache stampede?

Caching comes up in almost every round.

ANSWER POINTS

- Cache-aside: read the cache, on a miss load from the DB and set with a TTL. Invalidate by deleting on write, or use versioned keys.
- Key includes tenant/scope + a hash of the filters. Never cache per-user data under a shared key.
- Stampede: a lock so only one request rebuilds the value (others wait or get stale data), stale-while-revalidate, TTL jitter.
- Don't cache: money balances used for decisions, or anything that must be read right after a write.

LIKELY FOLLOW-UPS

- Redis as a queue vs RabbitMQ: what do you lose?

BE-14 What is the transactional outbox pattern and why do you need it?

The main event-driven correctness question.

ANSWER POINTS

- Problem: 'save to DB, then publish to RabbitMQ' can fail halfway. You get a DB write with no event, or an event for a rolled-back write.
- Fix: in the same DB transaction, insert the event into an `outbox` table. A relay process polls it (or reads the binlog with CDC, for example Debezium), publishes, and marks rows as sent.
- This gives at-least-once delivery, so consumers dedupe by event ID (inbox table with a unique key).

LIKELY FOLLOW-UPS

- How do you keep per-work-order ordering? (partition or route by work-order ID)

BE-15 RabbitMQ vs Kafka: when would you choose each?

Both are named in the job post.

ANSWER POINTS

- RabbitMQ: a broker that routes messages to queues; a message is removed after ack. Good for task queues, flexible routing, per-message retries/DLQ, moderate throughput.
- Kafka: a partitioned, replicated log; messages stay for the retention period and consumers track offsets. Good for high throughput, replay, event sourcing, many independent consumers, ordering per partition.
- For Field Nation-style commands (send notification, generate invoice) use RabbitMQ. For an event stream feeding analytics, search indexing and audit use Kafka.

LIKELY FOLLOW-UPS

- How does a Kafka consumer group scale, and what limits it? (number of partitions)

BE-16 Service A calls service B synchronously and B is slow. What protects A?

Microservice resilience.

ANSWER POINTS

- Timeouts on every call (never the default of no timeout).
- Retries only for idempotent calls, with exponential backoff + jitter and a cap.
- Circuit breaker: after N failures, fail fast for a cooldown period.
- Bulkheads: separate connection pools per dependency.
- Fallback: cached or partial response. Or move the call to async messaging if the user does not need the result immediately.

LIKELY FOLLOW-UPS

- How do retries make an outage worse? (retry storms)

BE-17 Liveness vs readiness probes, and how does a Node service shut down gracefully in Kubernetes?

Kubernetes is in the post.

ANSWER POINTS

- Readiness fails → the pod is removed from Service endpoints and gets no traffic (for example while warming up or when the DB is down). Liveness fails → the kubelet restarts the container (use it only for a stuck process).
- Mistake: checking the DB in liveness means a DB outage restarts every pod in a loop.
- Shutdown: pod gets SIGTERM → stop accepting new requests, finish in-flight ones, close DB/AMQP connections, exit before terminationGracePeriodSeconds (default 30 s). A short preStop sleep helps endpoints update first.

LIKELY FOLLOW-UPS

- Requests vs limits for CPU and memory? What is OOMKilled?

BE-18 What are the four golden signals, and how would you trace one request across three services?

Observability / APM is in the post.

ANSWER POINTS

- Latency (p50/p95/p99, not the average), traffic, errors, saturation (CPU, memory, queue depth, DB connections).
- Tracing: create a trace ID at the edge and pass it in HTTP headers (W3C traceparent) and in message headers. Each service logs it and records spans. OpenTelemetry → Jaeger/Tempo/Datadog.
- Logs are structured JSON with trace_id, tenant_id and user_id, so you can go from a metric spike to the trace to the logs.

LIKELY FOLLOW-UPS

- Why is average latency misleading?

BE-19 How do you test a service that writes to MySQL and publishes to RabbitMQ?

TDD and testing depth.

ANSWER POINTS

- Unit: domain logic with repository and publisher interfaces mocked.
- Integration: real MySQL + RabbitMQ in Docker (Testcontainers or docker-compose in CI). Assert the row and the published message.

- Contract tests (for example Pact) between producer and consumer, so a payload change breaks CI instead of production.
- Keep e2e tests few and slow-path only.

LIKELY FOLLOW-UPS

- How do you stop tests from sharing DB state?

BE-20 API authentication for third-party integrations: API keys, OAuth2 client credentials, or JWT?

Field Nation has a public API for buyers.

ANSWER POINTS

- API key: simple, good for server-to-server. Hash it at rest, scope it, make it rotatable.
- OAuth2 client credentials: short-lived access tokens, scopes, standard tooling. Better for a public platform.
- JWT is a token format, not a flow. It suits stateless verification across services, but revocation is hard, so keep them short-lived with refresh tokens.
- Add rate limits per client (token bucket in Redis) and return 429.

LIKELY FOLLOW-UPS

- How do you revoke a JWT before it expires?

C System design

Short estimation and architecture questions, then the 45-minute framework and six practice cases. Cover the reference notes and design each case on paper first.

The 45-minute framework

0-5 min

Clarify

Users, core use cases, what is out of scope. Ask about scale.

5-10

Estimate

Requests/s, storage, read:write ratio. Keep it rough.

10-15

API + data

3-5 endpoints, the main tables, and the status state machine.

15-30

High-level design

Boxes and arrows: gateway, services, DB, cache, queue, storage.

30-40

Deep dive

The hard part: concurrency, consistency, failure, hot spots.

40-45

Wrap up

Bottlenecks, monitoring and SLOs, what you would build next.

SD-01 Estimate the load: 1M work orders a year, 100k technicians. What dominates the traffic?

Estimation checks judgement. The numbers come from your dossier; confirm them.

ANSWER POINTS

- $1\text{M}/\text{year} \approx 2,740 \text{ per day} \approx 0.03 \text{ per second}$ on average. Even a $10\times$ peak is tiny for writes.
- Traffic comes from reads (technician search and job feeds), status updates, check-in pings, photo uploads, notifications and webhook fan-out: perhaps 20-50 events per work order.
- Storage: photos dominate. $1\text{M} \times 10 \text{ photos} \times 2 \text{ MB} \approx 20 \text{ TB}$ a year $\rightarrow$ S3 + CDN, never the DB.
- Conclusion: the hard parts are correctness, integrations and search, not raw QPS. Say this out loud.

LIKELY FOLLOW-UPS

- Where would you need read replicas first?

SD-02 Monolith or microservices for a new feature? How do you decide service boundaries?

They are mid-migration and want judgement, not buzzwords.

ANSWER POINTS

- Split by business capability with its own data (work orders, dispatch, payments, notifications), not by technical layer.
- A good boundary means few synchronous calls across it and one owner for each piece of data.
- Start in the monolith when the domain is unclear. Extract when scaling, deploy frequency or team ownership demands it.
- Costs of microservices: network failures, eventual consistency, tracing, operational load.

LIKELY FOLLOW-UPS

- How do two services share data they both need? (events + local read models, not a shared DB)

SD-03 How does a saga handle 'approve work → charge buyer → pay technician' across services?

Distributed transactions in their payment flow.

ANSWER POINTS

- No 2PC. Each step is a local transaction that publishes an event.
- Orchestration: a Payment Orchestrator tracks state and calls each step; if one fails it runs compensations (refund the buyer, reverse the ledger entry).
- Choreography: services react to each other's events. Simpler at first, harder to follow later.
- Every step must be idempotent and retryable, and the saga state is stored so it survives a crash.

LIKELY FOLLOW-UPS

- The technician payout fails permanently. What does the user see?

Work-order dispatch and matching

Buyers publish work orders. Route each one to suitable technicians by skill, distance, rating and availability. Technicians request or accept, and exactly one gets assigned.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- Functional: publish WO, find candidate techs, notify them, tech requests, buyer or auto-assign, confirm.
- Non-functional: assignment must never double-book; search p95 < 300 ms; notifications within seconds.

MAIN FLOW

- Publish → outbox event WorkOrderPublished → Matching service queries candidates → ranks them → Notification service fans out push/SMS.
- Assign with a conditional UPDATE on status + version (BE-10) → WorkOrderAssigned event → calendar, notifications, buyer webhook.

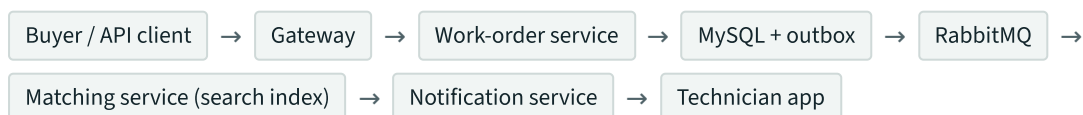
DATA MODEL

- work_orders (id, buyer_id, status, location POINT, skills, schedule_start, pay, version)
- providers (id, location, skills, rating, active)
- provider_availability, work_order_requests (wo_id, provider_id, status, UNIQUE)
- Spatial index or geohash column; Elasticsearch/OpenSearch for combined geo + skill + rating search

DEEP-DIVE POINTS

- Ranking: a score = weights × (distance, rating, completion rate, past work with this buyer). Keep it tunable per buyer.
- Hot spots: many techs request one urgent WO. The DB row lock handles correctness; rate-limit requests.
- Fairness: don't always notify the same top 10; rotate or batch in waves (first 20, widen the radius after 10 min).

ARCHITECTURE



Notifications and outbound webhooks

Send email, SMS, push and in-app notifications on work-order events, and deliver signed webhooks to buyer systems reliably.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- At-least-once, deduplicated, user preferences and quiet hours respected, templates per locale.
- One failing buyer endpoint must not delay other buyers.

DATA MODEL

- notification_preferences (user, channel, event_type, enabled)
- notifications (id, user, event_id, channel, status, attempts) UNIQUE(event_id, user, channel)
- webhook_endpoints (buyer_id, url, secret, events, status); webhook_deliveries (event_id, endpoint_id, attempt, status_code, next_retry_at)

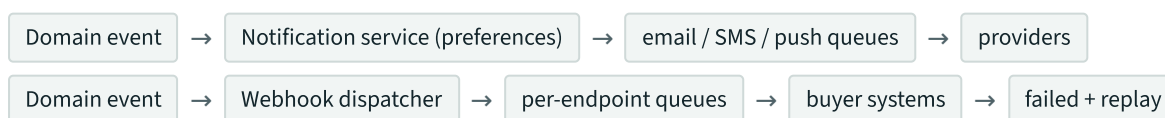
MAIN FLOW

- Domain event → Notification service → expand to (user × channel) → per-channel queues → provider adapters (SES, Twilio, FCM).
- Webhooks: per-endpoint queue → HMAC-signed POST → 2xx marks done; otherwise backoff + jitter; failed after 24 h → dashboard + replay.

DEEP-DIVE POINTS

- Priority lanes: urgent (tech en route) vs digest.
- Provider failover: second SMS provider behind a circuit breaker.
- Metrics: delivery latency per channel, failure rate per endpoint, queue depth.

ARCHITECTURE



Technician check-in, offline mobile and deliverables

A React Native app lets technicians check in and out with GPS and time, work offline on site, and upload photos and signatures as proof of work.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- Works with no signal and syncs later without duplicates.
- Evidence can't be faked easily; photos can be large.

MAIN FLOW

- App writes to a local queue (SQLite) with a `client_event_id` → syncs when online → server upserts by `client_event_id` (idempotent).
- Photos: the app requests a pre-signed S3 URL → uploads directly → confirms → a worker makes thumbnails and checks EXIF.

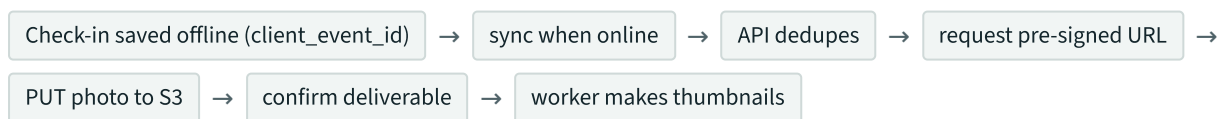
DATA MODEL

- `time_logs` (`wo_id`, `provider_id`, `type`, `lat`, `lng`, `accuracy`, `device_time`, `server_time`, `client_event_id` UNIQUE)
- `deliverables` (`wo_id`, `s3_key`, `sha256`, `taken_at`, `lat`, `lng`)

DEEP-DIVE POINTS

- Conflicts: server state wins for status; append-only for logs. Keep both device and server time.
- Anti-spoofing: geofence radius, impossible speed, mock-location flag (your CV-08 story).
- Battery: no continuous tracking, only event-based location.

ARCHITECTURE



Work approval and technician payouts

After the buyer approves the work, charge the buyer and pay the technician. Money must never be lost or paid twice.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- Exactly-once effect (idempotency), auditable, reconciles with the payment provider.
- Handles disputes and partial payments.

MAIN FLOW

- Approved → saga: charge buyer (idempotent key) → ledger entries → payout batch → provider webhook confirms → mark paid.
- Nightly reconciliation compares the ledger with provider settlement files and raises alerts.

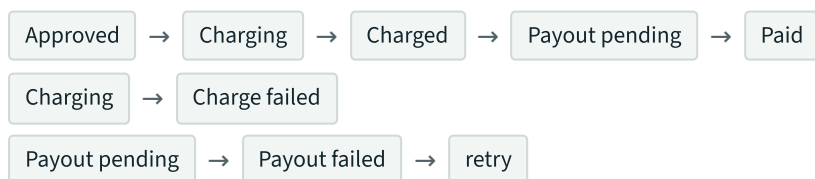
DATA MODEL

- Double-entry ledger: ledger_entries (txn_id, account, debit, credit) with every transaction summing to zero
- payouts (id, provider_id, amount, currency, status, idempotency_key UNIQUE, provider_ref)

DEEP-DIVE POINTS

- Never update balances in place; derive them from the ledger.
- Decimal or integer minor units, never floats.
- State machine for payout: pending → processing → paid / failed → retry or manual.

ARCHITECTURE



Extract a PHP module into a Node.js service

Move one domain (say, notifications or work-order search) out of the PHP/MySQL monolith into a Node.js service on Kubernetes with zero downtime.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- No downtime, easy rollback, data stays consistent during the transition.

MAIN FLOW

- 1. Put a routing layer (gateway/Nginx/Ingress) in front. 2. Monolith publishes events via outbox. 3. Node service builds its own data. 4. Shadow traffic and compare responses. 5. Move traffic with a flag, per tenant or by percentage. 6. Make the monolith read-only for that domain, then delete the old code.

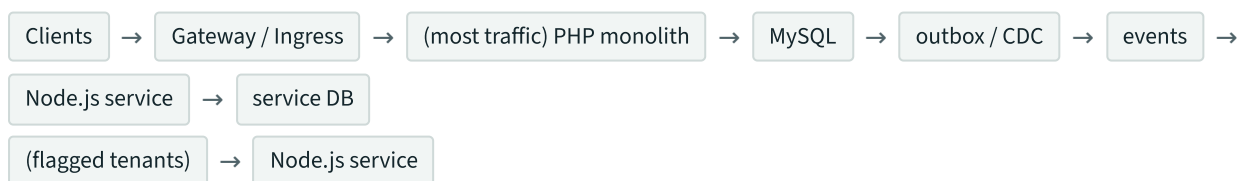
DATA MODEL

- The new service owns its tables. Until cutover it builds them from monolith events (outbox or CDC).

DEEP-DIVE POINTS

- Avoid dual writes from app code; use outbox or CDC.
- Contract tests keep the API identical for clients.
- This is your CV-02 story. Tell it as 'I did this, and here is how I would do it at Field Nation.'

ARCHITECTURE



Multi-tenant platform with noisy neighbours

Enterprise buyers vary hugely in volume. Stop one big buyer's bulk import or report from slowing down everyone else.

Your design: set a 40-minute timer and sketch it here first.

REFERENCE NOTES

REQUIREMENTS

- Fair resource use, per-tenant limits, isolation of data and performance.

MAIN FLOW

- Per-tenant rate limits at the gateway (token bucket in Redis).
- Separate queues or weighted fair scheduling per tenant for heavy jobs.
- Reports run on read replicas or a precomputed store, never on the primary.

DATA MODEL

- Shared schema with tenant_id vs DB per tenant vs a hybrid (big tenants get their own DB).

DEEP-DIVE POINTS

- Metrics labelled by tenant to spot the noisy one.
- Your Sokrio landlord/tenant design is the story; say what you would change at a larger scale.

ARCHITECTURE



03 How to answer

A technical question

1. Give the direct answer in one sentence. 2. Explain the mechanism. 3. Give a Sokrio example with a number. 4. Name the trade-off or the case where it breaks. Then stop and let them ask the follow-up.

A CV question

Situation → what *you* did → result with a number → what you'd change. Say "I", not "we", for your own part. If a claim on the CV is broader than what you did, narrow it yourself before they do.

When you don't know

"I haven't used that in production. Here's how I'd reason about it..." and then reason from first principles. Guessing confidently and being wrong costs more than saying you haven't done it.

Numbers to remember

99.9% over 30 days	43.2 minutes of error budget
99.99% over 30 days	4.3 minutes
1 day	86,400 s, about 10^5
1M events per day	about 12 per second on average
1M work orders per year	about 2,740 per day (dossier figure, confirm it)
Memory / SSD read / same-region round trip	about 100 ns / 100 μ s / 0.5–1 ms
Kubernetes termination grace	30 s by default

Questions to ask them

- Which services have already moved from PHP to Node, and which one is next?
- Is RabbitMQ or Kafka in production, and for what?
- How do you define and review SLOs? Who owns on-call?
- What would a new engineer own in the first 90 days?